Lab: position computation

1. INTRODUCTION

The aim of this lab is to apply GNSS positioning algorithms, to study the main performance parameters, and to evaluate the impact of measurement errors or satellite failures.

The data used in the TD contains:

- Satellite position and clock offset. The coordinates are expressed in meters, in ECEF frames
  - CSV files: xsat.csv, ysat.csv, zsat.csv, bsat.csv
- Simulated measurements
  - CSV file: Csimu.csv

The receiver is static. Its reference position is:

$$\begin{aligned} x_{ref} &= 4628684.4674\ m \\ y_{ref} &= 119997.1175\ m \\ z_{ref} &= 4372110.0327\ m \\ b_{ref} &= b_0 + b_1 t_{simu}\ \mathrm{m} \\ b_0 &= c \times 50 \times 10^{-6}\ m \\ b_1 &= c \times 0.2 \times 10^{-9}\ s^{-1} \end{aligned}$$

$c = 299792458\ m/s$ is the speed of light in vacuum.

Remark: it is a current process to validate new estimation methods with geodetic stations whose position is very precisely known.

Satellite position data are computed from actual GPS navigation files

- Start: 2020-12-05T00:00:00 (GPS)
- End: 2020-12-05T00:29:59 (GPS)
- Time step: 1s (1800 epochs)
- Satellites: 9 GPS satellites PRN 10, 16, 18, 20, 23, 26, 27, 29, 31

The date and time parameters can be converted from calendar to GPS format:

- Calendar: 2020-12-05, from 00:00:00 to 00:29:59
- Week/TOW: 2134/(518400s to 520199s)
- DOY/TOD: 340/(0s to 1799s)

The measurements have been **simulated**. Basically, we have for the i-th satellite:

$$\begin{aligned} \rho_i(t_k) = &\sqrt{\left(x_i(t_k) - x_{sta}(t_k)\right)^2 + \left(y_i(t_k) - y_{sta}(t_k)\right)^2 + \left(z_i(t_k) - z_{sta}(t_k)\right)^2} \\ &+ b_{sta}(t_k) - b_i(t_k) + D_i(t_k) + \varepsilon_i(t_k) \end{aligned}$$

D represents small delays to account for the residual delays non-zero mean values.
$\varepsilon_i \sim \mathcal{N}(0, \sigma_{UERE})$ is the residual error, whose distribution is a normal centered distribution, with the same standard deviation σ_{UERE} for every satellite.

2. PRELIMINARIES

2.1. ECEF TO ENU TRANSFORMATION

The satellite and receiver coordinates are expressed in ECEF frame. On top of that, it is possible to attached to a point on the Earth (as a station for instance) a local reference frame built with:

- an axis eastwards (the East axis),
- an axis northwards (the North axis),
- an axis upwards (the Up axis),

as depicted on the Figure 1.

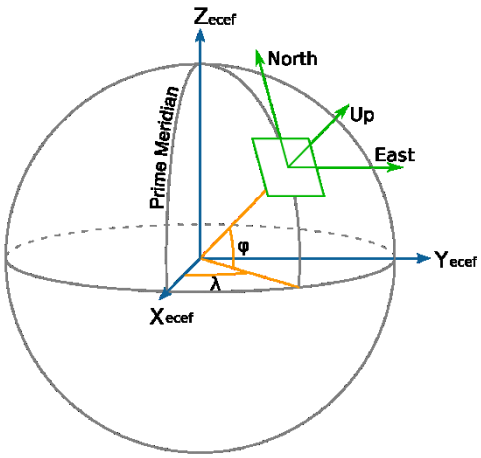

Figure 1 - Definition of the ENU reference frame (source: wikipedia)

The relative vector from the station to the satellite $\overrightarrow{p_{sat}} - \overrightarrow{p_{sta}}$ can then be expressed in the local East-North-Up reference frame of the station.

To do so, it is first mandatory to compute the rotation matrix from the ECEF reference frame to the local frame of the station. If the station as the coordinates $x_{sta}, y_{sta}, z_{sta}$ in the ECEF frame, its latitude and longitude can be computed as (assuming the Earth is a sphere):

$$\lambda_{sta} = \text{atan}\left(\frac{y_{sta}}{x_{sta}}\right), \varphi_{sta} = \text{atan}\left(\frac{z_{sta}}{\sqrt{x_{sta}^2 + y_{sta}^2}}\right) = \text{asin}\left(\frac{z_{sta}}{\sqrt{x_{sta}^2 + y_{sta}^2 + z_{sta}^2}}\right)$$

Be careful when computing the reference longitude, you will have to check the quadrant in which the angle is defined. To this end, most of the programming languages have an `atan2` function that performs this check. This is not needed for the reference latitude as this angle is defined in $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$ (although it is not false to use it).

From the latitude and the longitude of the station, the rotation matrix from the ECEF frame to the station local ENU reference frame is given by:

$$R_{ECEF \to ENU} = \begin{pmatrix} -\sin\lambda_{sta} & \cos\lambda_{sta} & 0 \\ -\sin\varphi_{sta}\cos\lambda_{sta} & -\sin\varphi_{ref}\sin\lambda_{sta} & \cos\varphi_{sta} \\ \cos\varphi_{sta}\cos\lambda_{sta} & \cos\varphi_{sta}\sin\lambda_{sta} & \sin\varphi_{sta} \end{pmatrix}$$

Note that the rotation matrix depends on the station for which the local East-North-Up frame is computed.

Once the rotation matrix is computed, the relative station-satellite vector can be rotated in the local frame. Its coordinates are noted $\Delta E_{sta \to sat}, \Delta N_{sta \to sat}, \Delta U_{sta \to sat}$ and are computed with the following formula:

$$\begin{pmatrix} \Delta E_{sta \to sat} \\ \Delta N_{sta \to sat} \\ \Delta U_{sta \to sat} \end{pmatrix}_{ENU} = R_{ECEF \to ENU} \begin{pmatrix} x_{sat} - x_{sta} \\ y_{sat} - y_{sta} \\ z_{sat} - z_{sta} \end{pmatrix}_{ECEF}$$

1) Compute the reference station longitude λ_{ref} and latitude φ_{ref}, and the rotation matrix $R_{ECEF \to ENU}$.
2) Compute the rotation matrix from the ECEF frame to the local ENU frame of the reference station.

2.2. SATELLITE ELEVATION

The elevation is the angle from the local horizontal plane to the receiver-to-satellite line of sight. The azimuth is the angle from the North direction to the projection of the receiver-to-satellite line of sight on the horizontal plane.

$$El = \text{asin}\frac{\Delta U_{sta \to sat}}{d_{sat}}$$

$$Az = \text{atan}(\Delta E_{sta \to sat}, \quad)$$

`delta_N_sta_sat`

$$d_{sat} = \sqrt{\Delta E_{sta\to sat}^2 + \Delta N_{sta\to sat}^2 + \Delta U_{sta\to sat}^2}$$

3) Compute the satellites azimuth and elevation angles versus time.
4) Run the given script completed with the functions defined in the previous questions and draw the skyplot. Analyse it.

2.3. DOP COMPUTATION

The cosine director matrix is computed as follow, with $(x_{sat,j}, y_{sat,j}, z_{sat,j})$ being the j-th satellite coordinates in the ECEF reference frame.

$$H = H_k = \begin{bmatrix} \frac{x_{sta} - x_{sat,1}}{r_1^k} & \frac{y_{sta} - y_{sat,1}}{r_1^k} & \frac{z_{sta} - z_{sat,1}}{r_1^k} & 1 \\ \vdots & \vdots & \vdots & \vdots \\ \frac{x_{sta} - x_{sat,i}}{r_i^k} & \frac{y_{sta} - y_{sat,i}}{r_i^k} & \frac{z_{sta} - z_{sat,i}}{r_i^k} & 1 \\ \vdots & \vdots & \vdots & \vdots \\ \frac{x_{sta} - x_{sat,n}}{r_n^k} & \frac{y_{sta} - y_{sat,n}}{r_n^k} & \frac{z_{sta} - z_{sat,n}}{r_n^k} & 1 \end{bmatrix}$$

$$r_i = \sqrt{\left(x_{sta} - x_{sat,i}\right)^2 + \left(y_{sta} - y_{sat,i}\right)^2 + \left(z_{sta} - z_{sat,i}\right)^2}$$

From that matrix, it is possible to compute the matrix :

$$M_{xyz} = (H^T H)^{-1} = \left[m_{i,j}^{xyz}\right]_{i,j\in[1,4]}$$

The various Dilutions of Precision can be computed:

- the Geometric Dilution of Precision (GDOP): $GDOP = \sqrt{m_{11}^{xyz} + m_{22}^{xyz} + m_{33}^{xyz} + m_{44}^{xyz}}$,
- the Position Dilution of Precision (PDOP): $PDOP = \sqrt{m_{11}^{xyz} + m_{22}^{xyz} + m_{33}^{xyz}}$,
- the Time Dilution of Precision (TDOP): $TDOP = \sqrt{m_{44}^{xyz}}$,

with m_{ii} being the diagonal elements of the matrix M_{xyz}.

By using the rotation matrix from the ECEF reference frame to the local reference frame $R_{ECEF\to ENU}$, the M_{xyz} matrix can be rotated in the local frame: $M_{enu} = R_{xyz\to enu} M_{xyz} R_{xyz\to enu}^T$. Because the last column of the H matrix is 1 (it correspond to the partial derivative w.r.t. the receiver clock offset), the matrix to rotate the M_{xyz} in M_{enu} is given by:

$$R_{xyz\to enu} = \begin{bmatrix} R_{ECEF\to ENU} & 0_{3\times 1} \\ 0_{1\times 3} & 1 \end{bmatrix}$$

The terms of the M_{enu} matrix are denoted $\left[m_{i,j}^{enu}\right]_{i,j\in[1,4]}$ and the local dilutions of precision can be derived:

- the Horizon Dilution of Precision (HDOP): $HDOP = \sqrt{m_{11}^{enu} + m_{22}^{enu}}$,
- the Vertical Dilution of Precision (VDOP): $VDOP = \sqrt{m_{33}^{enu}}$,

with m_{ii} being the diagonal elements of the matrix M_{enu}.

5) For each epoch of the simulation, compute the director cosine matrix H. From this matrix and the ECEF to ENU rotation matrix, compute the PDOP, TDOP, HDOP and VDOP.

6) What is the impact on the DOP values of the loss of several of the most elevated satellites? And that of the least elevated satellites? To answer this question, change the list of the satellite that are taken into account to compute the DOP values.

3. POSITION COMPUTATION

3.1. LEAST SQUARES ALGORITHM IMPLEMENTATION

At each simulation epoch t_i, the least squares estimation (LSE) algorithm has to be executed. In the following, we will denote $\rho(t_i)$ the measurements vector at epoch i for all the satellites. This vector can be decomposed in the measurements of each satellite:

$$\rho(t_i) = \begin{bmatrix} \rho_1(t_i) \\ \vdots \\ \rho_{n_S}(t_i) \end{bmatrix},$$

with n_S the number of satellites. Each pseudo range measurement $\rho_j(t_i)$ of satellite j is modelled as follows:

$$\rho_j(t_i) = \sqrt{\left(x_{sta}(t_i) - x_j(t_i)\right)^2 + \left(y_{sta}(t_i) - y_j(t_i)\right)^2 + \left(z_{sta}(t_i) - z_j(t_i)\right)^2} + b_{sta}(t_i) - b_j(t_i) + D_j + \varepsilon_j(t_i).$$

$[x_j(t_i) \quad y_j(t_i) \quad z_j(t_i)]^T$ and $b_j(t_i)$ are respectively the coordinates and the clock offset of the j-th satellite at time t_i and $p(t_i) = [x_{sta}(t_i) \quad y_{sta}(t_i) \quad z_{sta}(t_i) \quad b_{sta}(t_i)]^T$ is the vector of the unknown reference station position and clock offset.

It is thus possible to define a function f_j for each satellite such that: $\rho_j(t_i) = f_j\big(p(t_i)\big)$. Defining the vector function:

$$f\big(p(t_i)\big) = \begin{bmatrix} f_1\big(p(t_i)\big) \\ \vdots \\ f_{n_S}\big(p(t_i)\big) \end{bmatrix},$$

the model equation is rewritten in vector form:

$$\rho(t_i) = f\big(p(t_i)\big),$$

with $p(t_i)$ unknown. This equation is solved at each epoch by an iterative least squares algorithm. This algorithm is an iterative process. Beginning by a first guess of the solution $p^{(0)}(t_i)$, it is possible to compute as a k-th iteration a solution $p^{(k)}(t_i)$ from the previous solution $p^{(k-1)}(t_i)$.

This paragraph details how to compute the solution at the k-th iteration $p^{(k)}(t_i)$ from the solution computed at the $k-1$-th iteration. Assuming that $p^{(k-1)}(t_i)$ is known, the k-th iteration $p^{(k)}(t_i)$ is written as:

$$p^{(k)}(t_i) = p^{(k-1)}(t_i) + \Delta p^{(k-1)}(t_i),$$

with $\Delta p^{(k-1)}(t_i)$ supposed to be small. With this assumption, it is possible to rewrite the model equation with a first order Taylor development:

$$\rho(t_i) = f\left(p^{(k)}(t_i)\right) = f\left(p^{(k-1)}(t_i) + \Delta p^{(k)}(t_i)\right) = f\left(p^{(k-1)}(t_i)\right) + H^{(k-1)}(t_i)\,\Delta p^{(k-1)}(t_i),$$

with $H^{(k-1)}(t_i)$ being the Jacobian matrix of the function $p \mapsto f(p)$ evaluated at $p = p^{(k-1)}(t_i)$:

$$H^{(k-1)}(t_i) = \left.\frac{\partial f(p)}{\partial p}\right|_{p_{k-1}}$$

$H^{(k-1)}(t_i)$ is also the direction cosine matrix at time t_i for the k-th iteration. In the equation above, $f\left(p^{(k-1)}(t_i)\right)$ is known from the previous iteration. This term can be put in the left hand side, which leads to:

$$\Delta\rho^{(k-1)}(t_i) = \rho(t_i) - f\left(p^{(k-1)}(t_i)\right) = H^{(k-1)}(\mathrm{t_i})\,\Delta p^{(k-1)}(t_i).$$

The quantity $\Delta\rho^{(k-1)}(t_i)$ is called the measurement residual, and is the difference between the true measurements and the measurements model evaluated at the previous iteration. The equation $\Delta\rho^{(k-1)}(t_i) = H^{(k-1)}(\mathrm{t_i})\,\Delta p^{(k-1)}(t_i)$ is a linear equation whose unknown is $\Delta p^{(k-1)}(t_i)$. It can be solved with the pseudo-inverse:

$$\Delta p^{(k-1)}(t_i) = \left(H^{(k-1)}(\mathrm{t_i})^T\, H^{(k-1)}(\mathrm{t_i})\right)^{-1} H^{(k-1)}(\mathrm{t_i})^T\,\Delta\rho^{(k-1)}(t_i),$$

and the k-th iteration solution reads: $p^{(k)}(t_i) = p^{(k-1)}(t_i) + \Delta p^{(k-1)}(t_i)$.

The least square algorithm is summarized as follows

```
Loop over each epoch t_i

     Initialization of the iteration process:
                    p^(0)(t_i) = [x^(0)(t_i)  y^(0)(t_i)  z^(0)(t_i)  b^(0)(t_i)]^T
```

For k-th iteration do:

$$\Delta\rho^{(k-1)}(t_i) = \rho(t_i) - f\left(p^{(k-1)}(t_i)\right)$$

$$\Delta p^{(k-1)}(t_i) = \left(H^{(k-1)}(\mathrm{t_i})^T\, H^{(k-1)}(\mathrm{t_i})\right)^{-1} H^{(k-1)}(\mathrm{t_i})^T \Delta\rho^{(k-1)}(t_i)$$

$$p^{(k)}(t_i) = p^{(k-1)}(t_i) + \Delta p^{(k-1)}(t_i)$$

Convergence control:

Stop iteration when $\left\|\Delta p^{(k-1)}(t_i)\right\| < threshold$ or after N iterations (to avoid infinite loops...)

Store $p^{(N)}(t_i),\ \Delta\rho^{(N)}(t_i)$

End of the loop over the epochs

At the first epoch, the receiver position and clock offset can be initialized with $(0, 0, 0, 0)$. Then, the receiver state vector at the k-th epoch can be initialized with the LSE output of the previous epoch.

At each epoch, use the receiver position after the LSE iteration to compute the measurement residual, and the DOP values.

7) Implement the least square estimation algorithm for the GNSS positioning problem.

3.2. ERROR COMPUTATION

8) Compute the position errors versus time. Are they in line with the DOP values?

3.3. IMPACT OF GEOMETRY

9) What is the impact on the position errors of the loss of several of the most elevated satellites? And that of the least elevated satellites?

10) Is that in line with the DOP values previously computed?

4. IMPACT OF SATELLITE FAILURE

GNSS satellite can have several type of failure impacting either the range measurement or the clock and ephemeris parameters. That kind of failure has a direct impact on the receiver position solution.

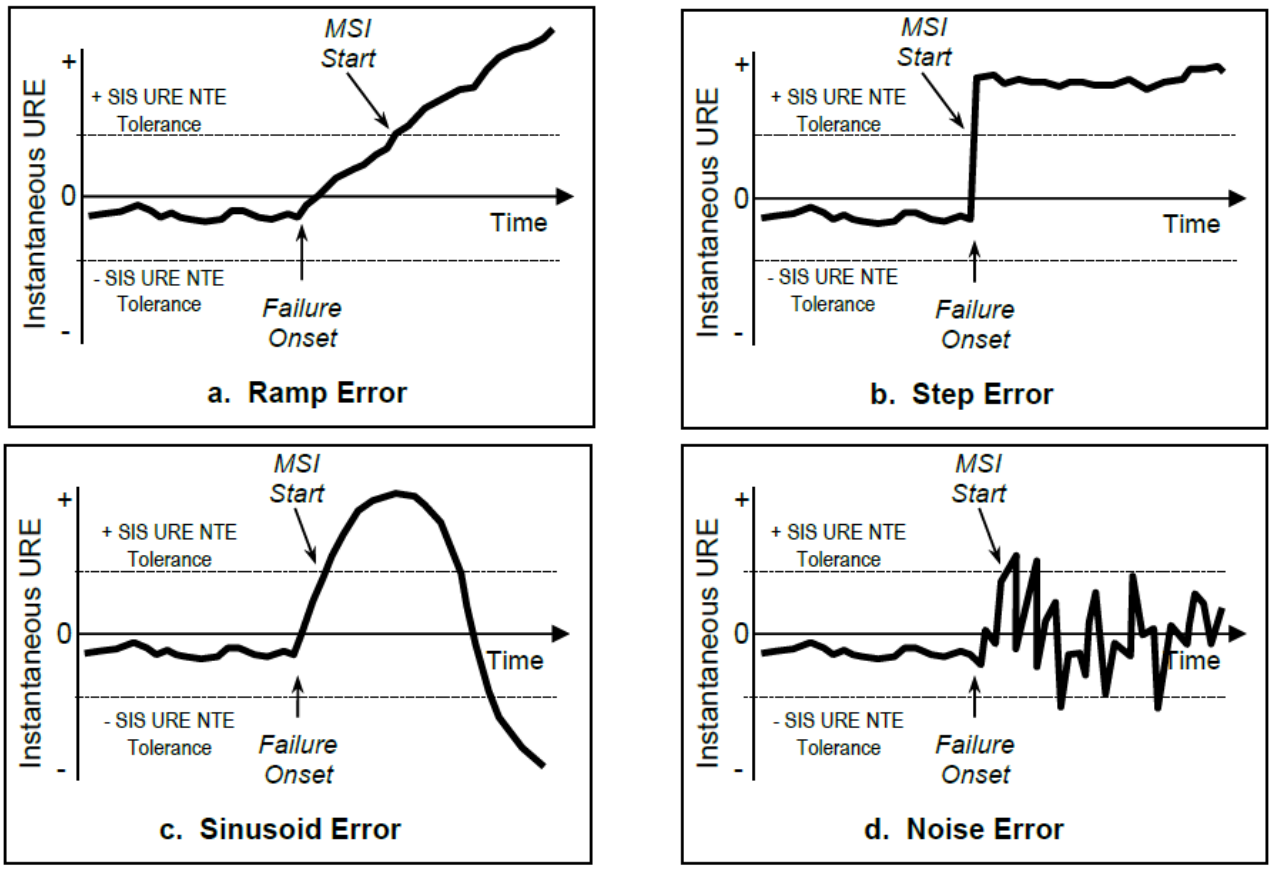

Figure A.5-1. Types of URE Effects

Figure 2: Different type of GPS failure (source GPS Standard Positioning Service Performance Standard 2020)

4.1. FAILURE SIMULATION

A satellite clock failure can be simulated on range measurement with the following equation (case of a clock abnormal drift):

$$\rho_{i,fail} = \rho_i + b_0 + b_1 \lfloor t_{simu} - t_{fail} \rfloor$$

where $\lfloor t_{simu} - t_{fail} \rfloor = \begin{cases} 0 \text{ if } t_{simu} < t_{fail} \\ t_{simu} - t_{fail} \text{ if } t_{simu} \geq t_{fail} \end{cases}$

11) Simulate the impact of a satellite clock failure for several satellites, and several b_1 drift values. How does the failure affect the position solution? Is the impact the same for low elevated satellites than for high elevated satellites?

4.2. TOWARD RAIM ALGORITHMS

A faulty satellite leads to faulty measurements for that satellite, and consequently, an erroneous position solution. When enough satellites are available, one can imagine to compute a solution without the faulty satellite, which is call *fault exclusion*. However, one needs to detect the faulty satellite first (*fault detection*).

Several FDE (*fault detection and exclusion*) algorithm exists. For example, RAIM (*Range Autonomous Integrity Monitoring*) is widely used in aviation.

Let's assume there is only one faulty satellite (the satellite can be freely chosen among the satellites list that has been provided for this exercise). The idea is to compare the so called 'all-in-view' solution (with all the satellites, even the faulty one) to all the possible fault mode solution. The i-th fault mode is the case where the faulty satellite is the i-th satellite: all the solutions containing the i-th satellite would provide erroneous solution, whereas the i-th solution, without the i-th satellite, would provide a good solution.

Two comparisons are possible

- Position solution comparison: the fault mode solutions are compared to the all-in-view solution (distance). When the fault is large enough, the i-th fault mode solution (without the faulty satellite) leads to a much better solution than the solution taking the faulty satellite into account.
- Residual comparison: when the fault is large enough, the i-th fault mode measurement residuals are small wrt. the all-in-view residuals.

12) Simulate a clock failure on 1 of the satellites, and compute the fault mode position solution and residuals. Compare the solution to the all-in-view solution. Can the faulty satellite always be detected? Does the position comparison method give the same results as the residual comparison method?